TALLER PRÁCTICO UNIVERSITARIO · 2 HORAS

IA Generativa y Sistemas Agénticos

Herramientas remotas y locales, desarrollo asistido y casos de aplicación real

Miguel Ángel Salinas Gancedo

uo34525@uniovi.es
masalinas.gancedo@gmail.com

Estructura del Taller

00 - 30 min

Fundamentos y Diferencias: ML clásico vs GenAI, LLMs multimodales y arquitectura de modelos remotos vs locales.

30 - 60 min

Ecosistema & Agentes: Del VibeCoding al SDD, protocolos MCP, configuración de Ollama y el TUI Antigravity.

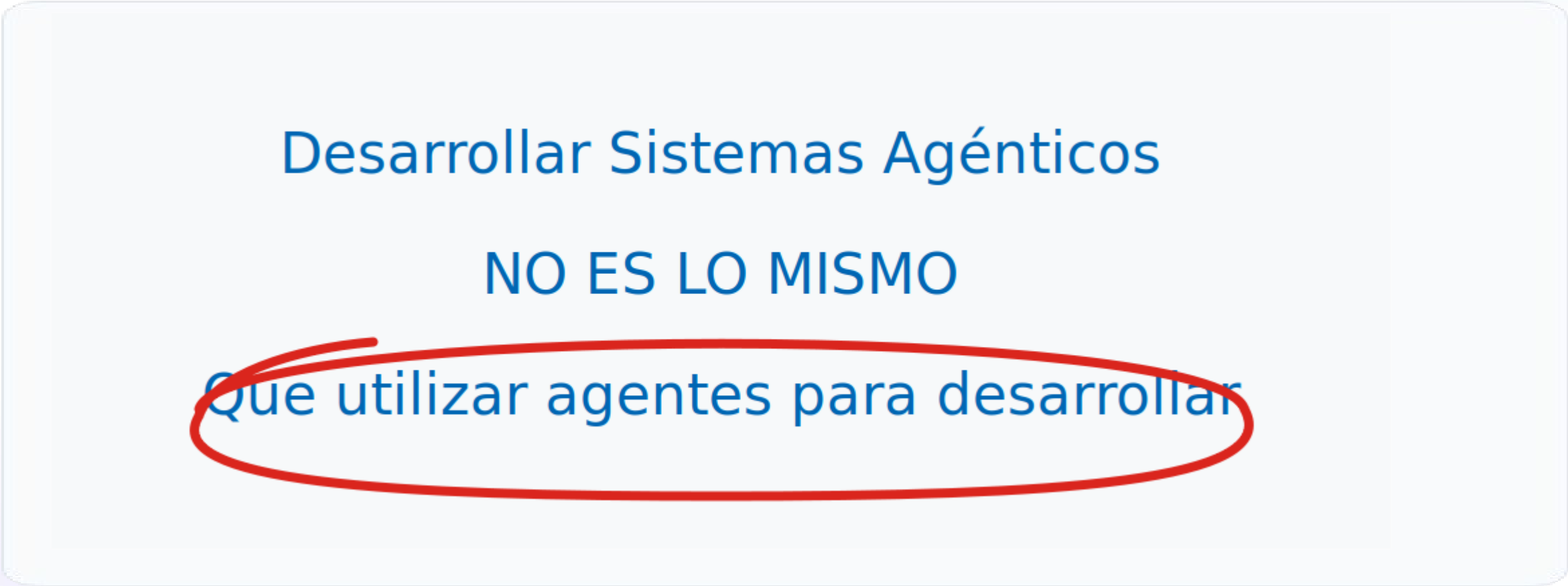
60 - 100 min

Demostraciones en Vivo: Ejecución de las 3 PoCs (Senoide Python, API FastAPI y simulación cinemática en MATLAB).

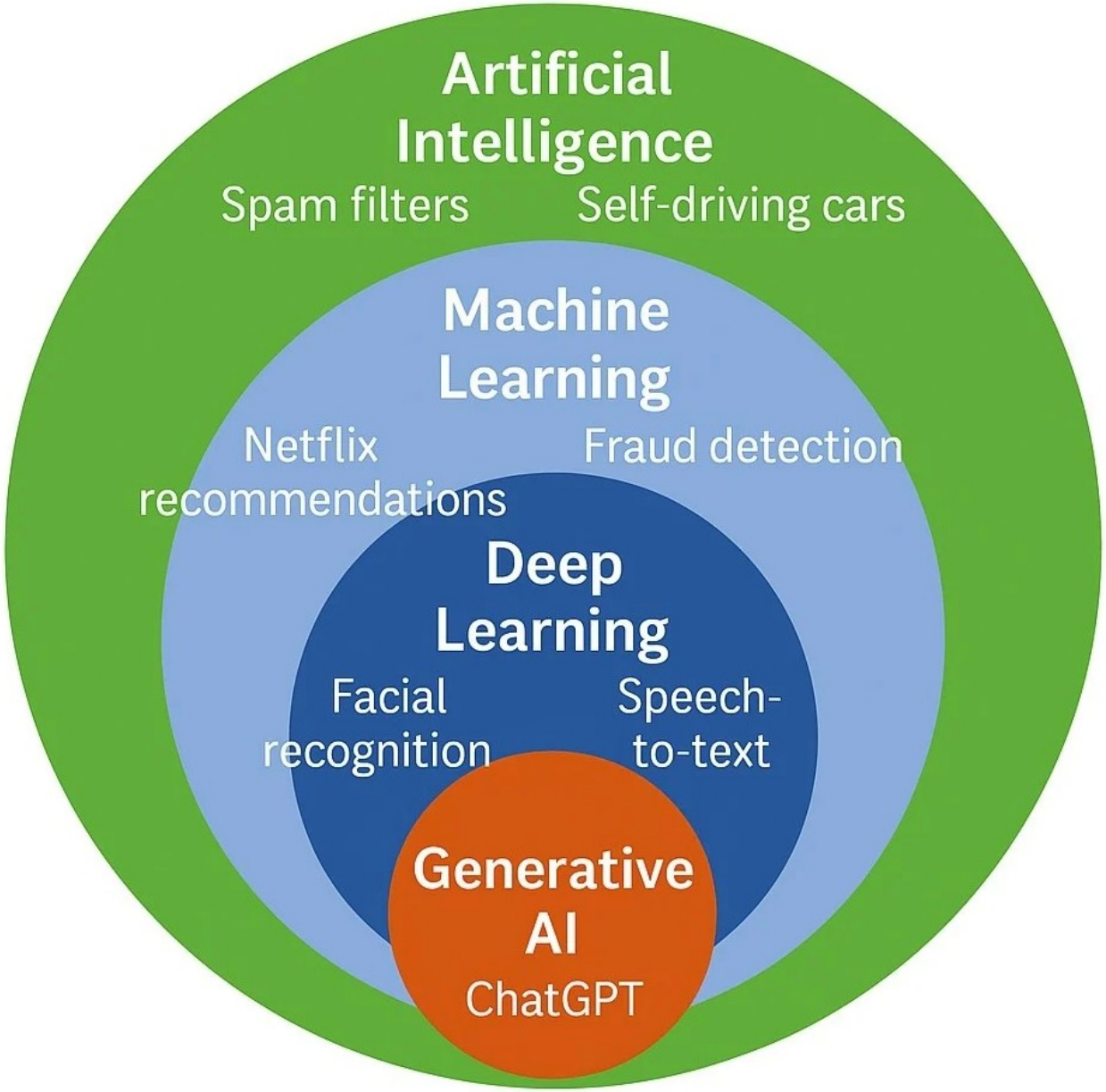
100 - 120 min

Laboratorio & Debate: Réplica guiada en equipos de alumnos, resolución de dudas técnicas y recomendaciones de carrera.

De que va este Taller

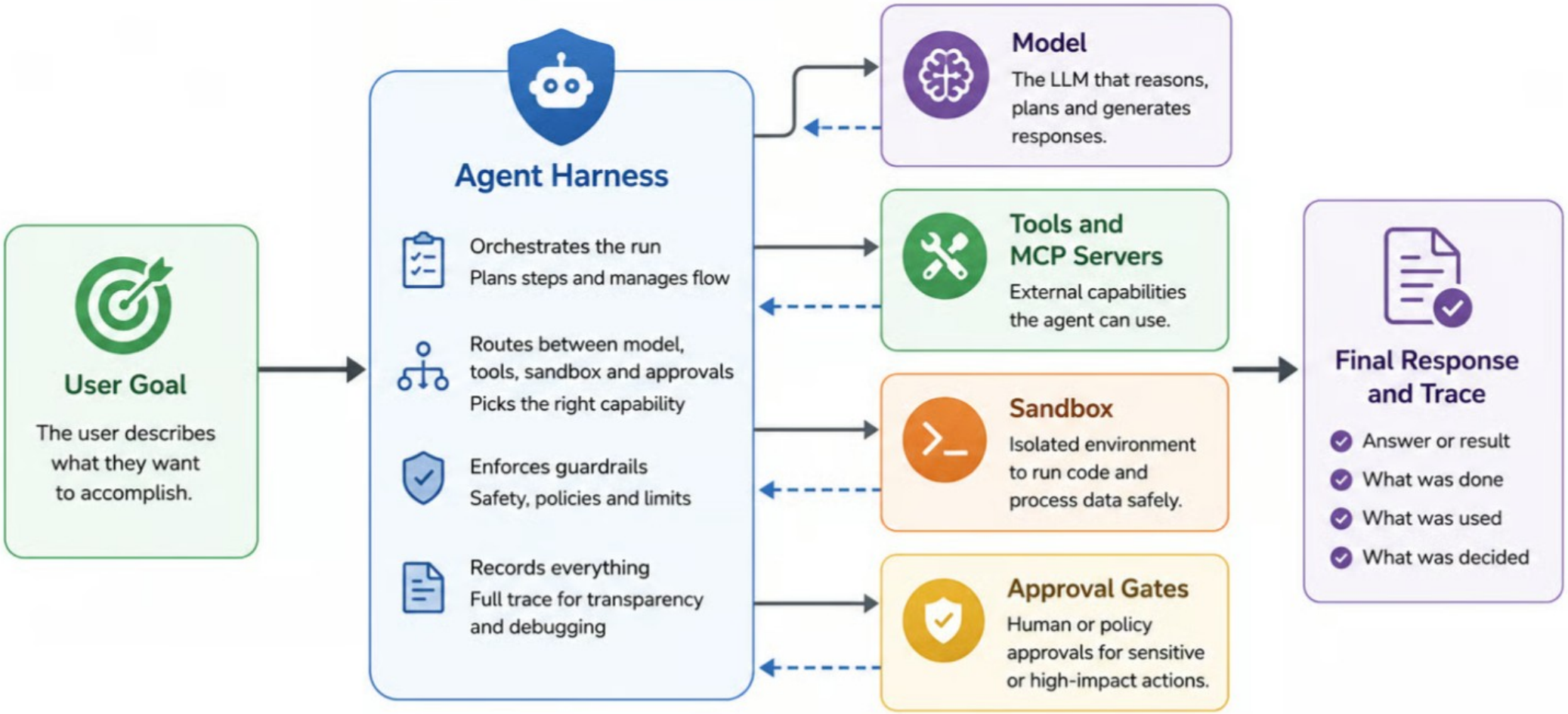


Contexto de la IA



Agent Harness

The harness orchestrates the agent run, connecting the model to tools, sandbox and approvals to deliver a safe, reliable result.



Harness sends requests / instructions

Components return results / events

Why a harness?

It keeps the agent reliable, safe and observable by controlling interactions, enforcing policies and capturing a full trace.

Modelos Remotos vs Modelos Locales

Criterio	LLM Remoto (Cloud API)	LLM Local (En Equipo Propio)
Capacidad de Razonamiento	Máxima (modelos frontera de cientos de miles de millones de parámetros).	Limitada por el tamaño de memoria VRAM (7B a 32B parámetros cuantizados).
Requisitos Hardware	Cualquier equipo con conexión a Internet y navegador o terminal ligero.	GPU dedicada de alta gama (Apple Silicon ≥16-36GB o NVIDIA RTX 4070+).
Privacidad & Datos	Las peticiones viajan al proveedor en la nube (requiere gobernanza).	100% privado y desconectado; confidencialidad total de código y patentes.
Costes y Planes	Modelos de suscripción o pago por token (descuentos pro para estudiantes).	Inversión única en equipo físico; coste variable de electricidad y refrigeración.

Ecosistema LLM: Remoto y Local



OpenAI / ChatGPT / Codex

Pioneros comerciales. Destacan por modelos de razonamiento (familia o1/GPT-4o) con amplia integración en plugins y herramientas de desarrollo.



Anthropic / Claude / Claude Code

Referentes mundiales en generación de código estructurado, comprensión lógica rigurosa y creadores del estándar abierto MCP (Model Context Protocol).



Google / Gemini / Antigravity

Líder en ventanas de contexto masivas (hasta 2M tokens) y procesamiento nativo multimodal (código, vídeo en directo, esquemas técnicos y audio).



Ollama & Open Weight

Motor de ejecución local por excelencia. Ejecuta modelos libres como Llama 3 (Meta), Qwen y DeepSeek de forma offline y con API compatible.

Alternativas de Ejecución para Alumnos

Local & Offline

Ollama + TUIs / IDE

Configuración: Instalar Ollama y descargar modelos como llama3:8b o qwen2.5-coder.

Conexión con Claude Code / TUI: Enrutamiento del endpoint local cambiando la variable de entorno de API base a localhost:11434.

Requisitos de Hardware: Exige al menos 16 GB de RAM unificada en Apple Silicon o GPU NVIDIA RTX serie 4000 de 8GB-12GB VRAM para respuesta fluida.

Gratuito / Híbrido

Cline, OpenRouter y Free Tiers

Cline (Extensión VS Code): Extensión agéntica dentro del editor que lee el workspace, genera archivos y ejecuta la terminal con aprobación humana previa.

OpenRouter & NVIDIA NIM: Capas gratuitas temporales que ofrecen acceso a modelos punteros sin pagar suscripciones fijas.

Estrategia Estudiantil: Desarrollar la arquitectura con APIs remotas de alta potencia y delegar refactorizaciones locales a Ollama.

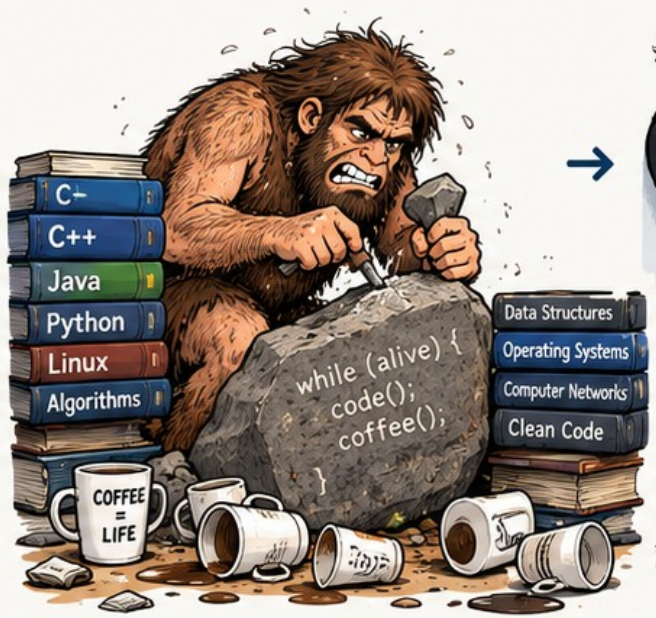
De Copiar Código a Sistemas Agénticos

Cómo los ingenieros han pasado de consultar respuestas en un navegador a coordinar agentes de software autónomos en terminal.

LA EVOLUCIÓN DEL PROGRAMADOR

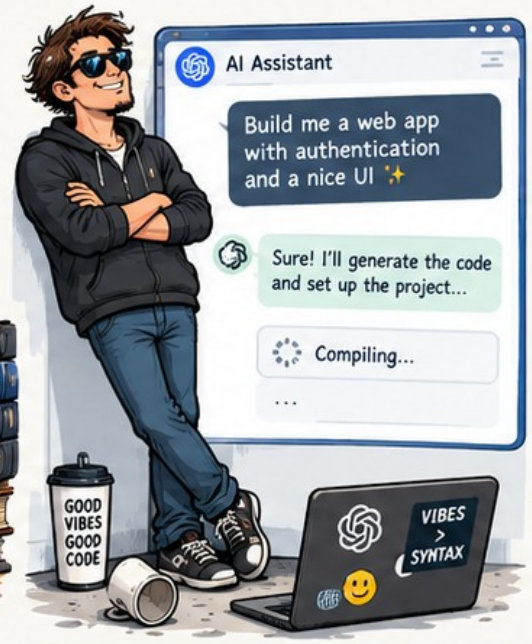
De picar código a dirigir agentes

1. PROGRAMADOR PRIMITIVO
(Mono encorvado)



- Código a mano (literalmente).
- Manuales pesados.
- Café, mucho café.
- Dolor, pero aprendizaje.

2. VIBE CODER
(Homínido semi-erguido)



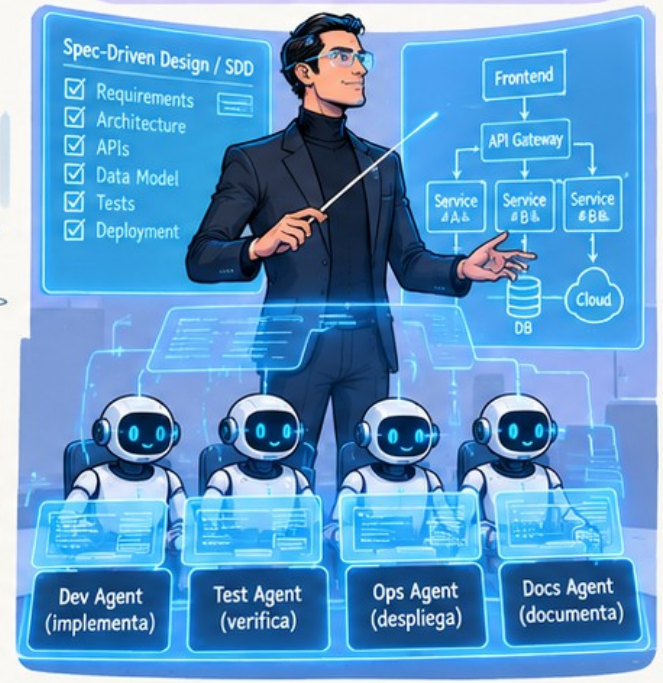
- Copia y pega en IA.
- Confía en la intuición.
- Cruza los brazos.
- Espera a que compile...
  (o que la IA lo arregle).

3. INGENIERO MODERNO
(Humano actual)



- Usa AGENTS.md.
- Guía y delega.
- Agentes que ejecutan.
- Más tiempo para pensar.
- Menos tiempo en lo repetitivo.

4. ARQUITECTO FUTURISTA
(Homo Sapiens tecnológico)



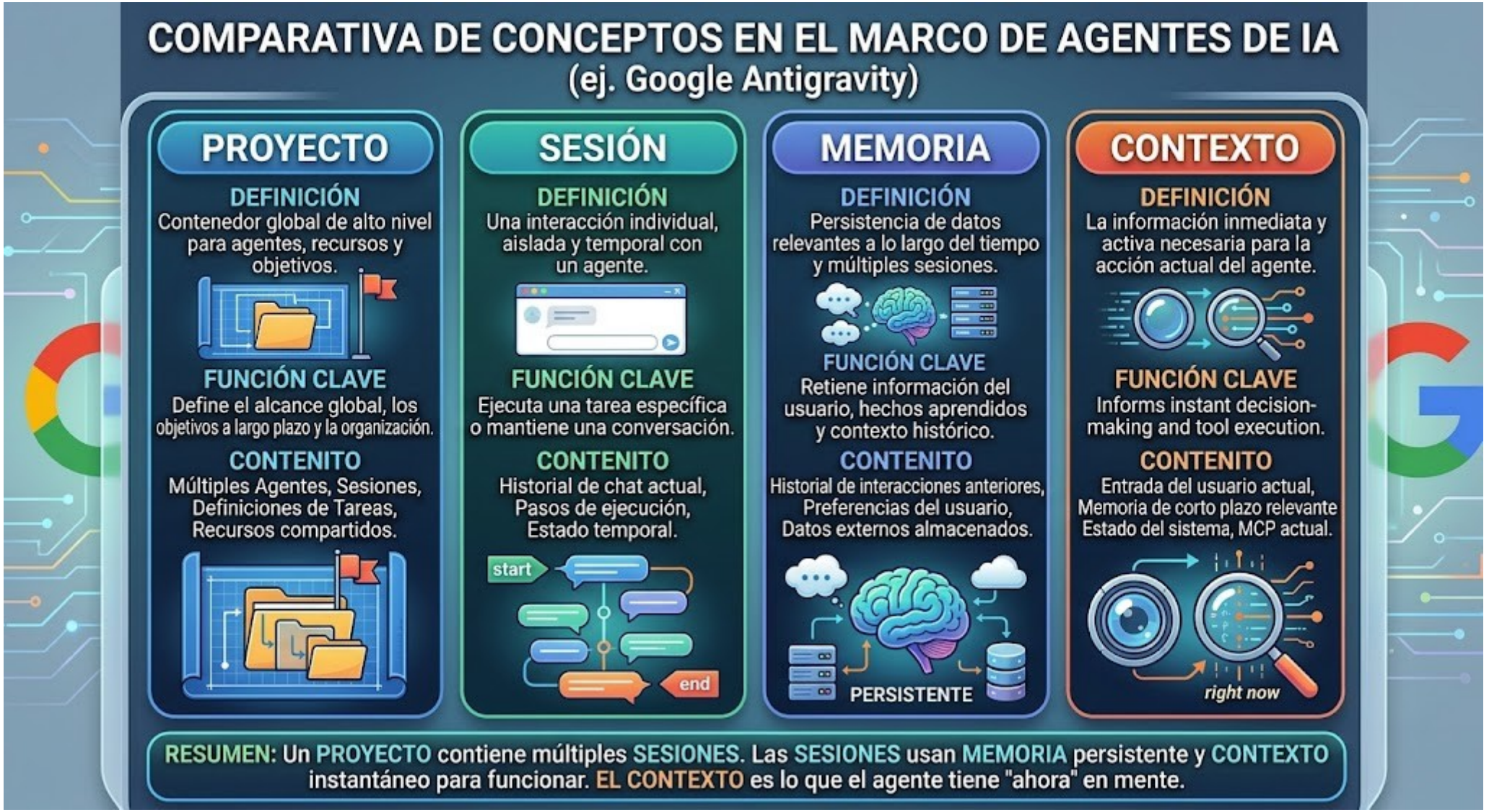
- No escribe código.
- Diseña, especifica, orquesta.
- Agentes alineados.
- Arquitectura, no solo features.
- El futuro ya es colaborativo.

MISMA PASIÓN · DIFERENTES HERRAMIENTAS · EL MISMO OBJETIVO: CONSTRUIR EL MAÑANA

Evolución Dev: Del Chatbox a SDD

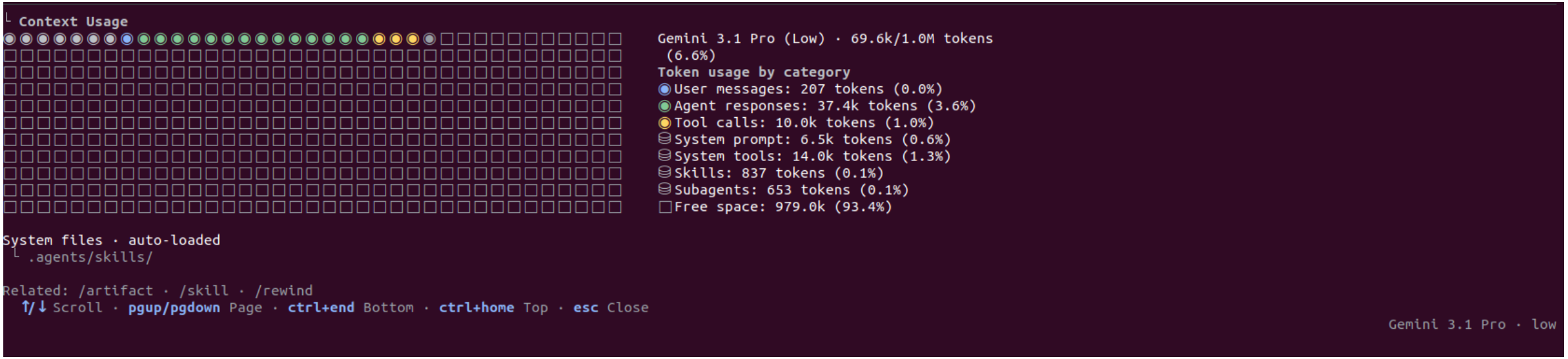
- **Fase 1 · VibeCoding en Chatbox:** Copiar y pegar fragmentos entre la interfaz web (ChatGPT/Claude/Gemini) y el editor. Método propenso a alucinaciones de contexto y pérdida de dependencias.
- **Fase 2 · Agentes de IA & TUIs (Terminal User Interfaces):** Herramientas como Claude Code, Codex y Antigravity que viven en tu consola. Leen el árbol de archivos, ejecutan tests, corrigen errores y usan herramientas externas.
- **Fase 3 · Protocolos MCP y Skills:** Los agentes se comunican con hardware, bases de datos o depuradores locales mediante estándares abiertos (Model Context Protocol), actuando sobre el entorno de trabajo real.
- **Fase 4 · Spec-Driven Design (SDD):** Especificación formal mediante archivos Markdown que definen ciclo de vida, restricciones y checklists. El agente opera con validación continua en lugar de prompts dispersos.

Algunos conceptos en la programación agéntica I

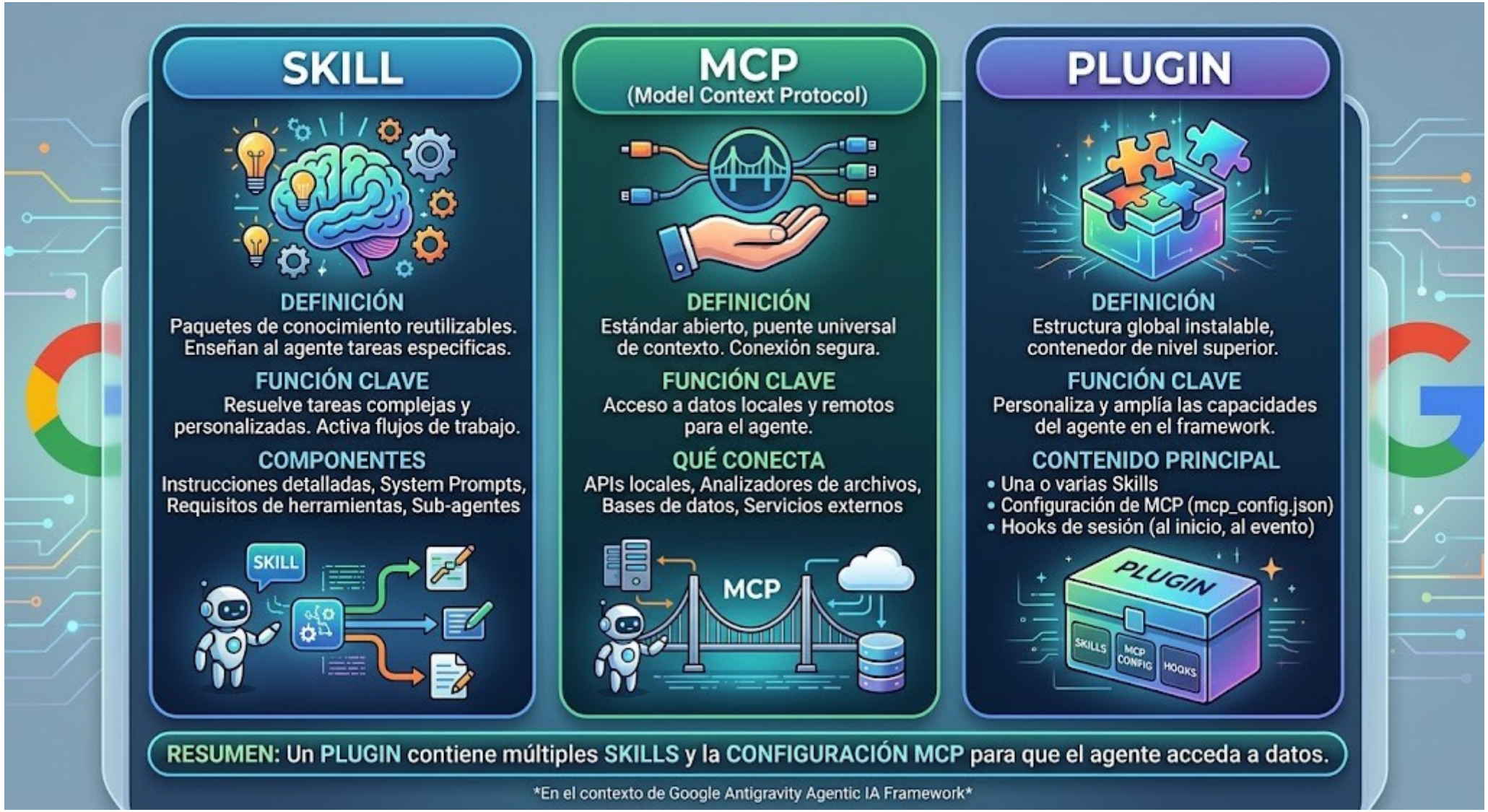


Algunos conceptos en la programación agéntica I

Como se ve el contexto desde el **agente antigravity**



Algunos conceptos en la programación agéntica II



Gemini para estudiantes



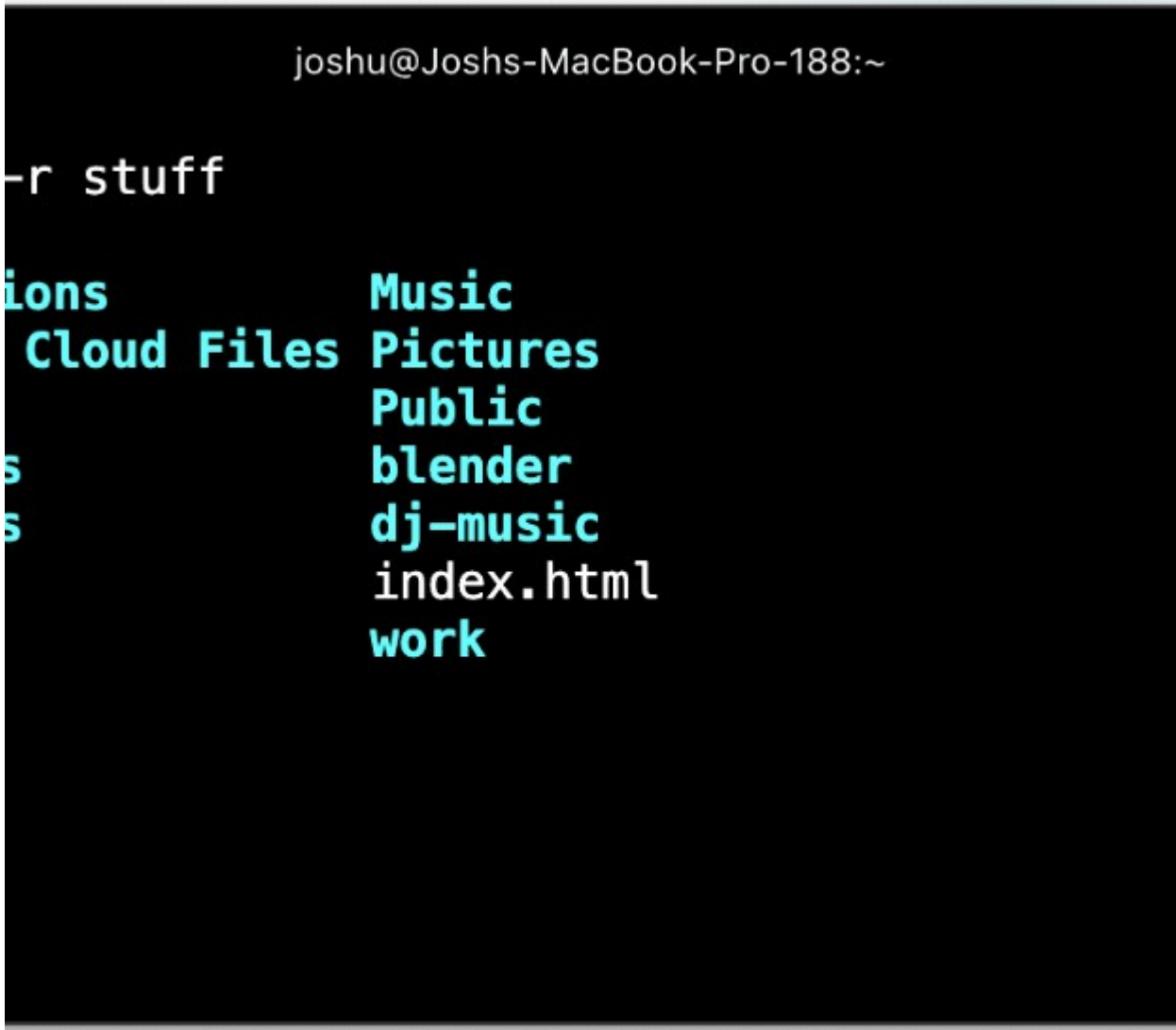
https://gemini.google/es/students/?hl=es

Antigravity y Desarrollo Autónomo

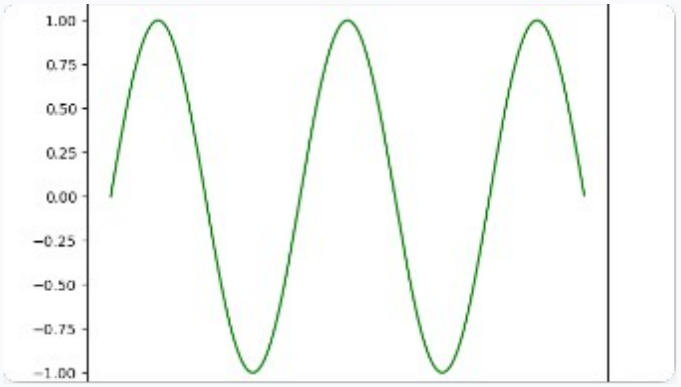
El Entorno Agéntico de Gemini

Antigravity es el entorno CLI/TUI para interactuar con Gemini Pro. Cuenta con versión de escritorio visual y una SDK modular para crear agentes conectados.

Google ofrece ventajas y precios reducidos a universitarios. Permite generar proyectos enteros desde especificaciones con ejecución autónoma de comandos seguros.

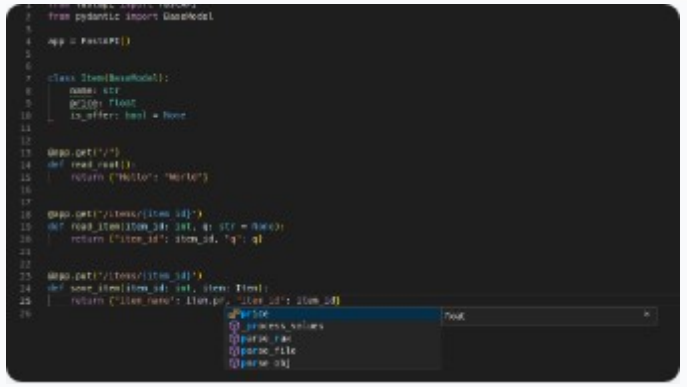


Pruebas de Concepto en el Taller



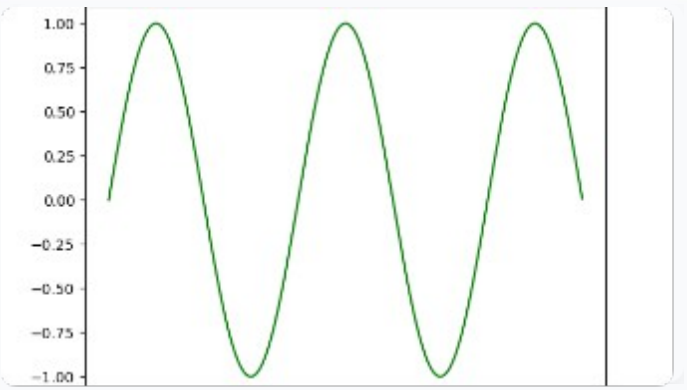
PoC 1 · Senoide en Python

Prompt rápido en TUI Antigravity para crear un script con NumPy y Matplotlib: parámetros interactivos de frecuencia, amplitud y diseño cromático en tiempo real.



PoC 2 · Microservicio FastAPI

Generación de un endpoint REST backend. Recibe el nombre del usuario y devuelve JSON estructurado con validación de tipos Pydantic y documentación Swagger automática.



PoC 3 · Cinemática MATLAB

Utilizar la herramienta de matlab para generar la senoide de la PoC 1, pero en este caso através del motor de Matlab y Python. Los scripts serán utilizando el lenguaje de matlab

Bonus: utilizar antigravity con modelos externos

1) Crear un fichero **AGENTS.md** en tu directorio raiz de proyecto con este texto:

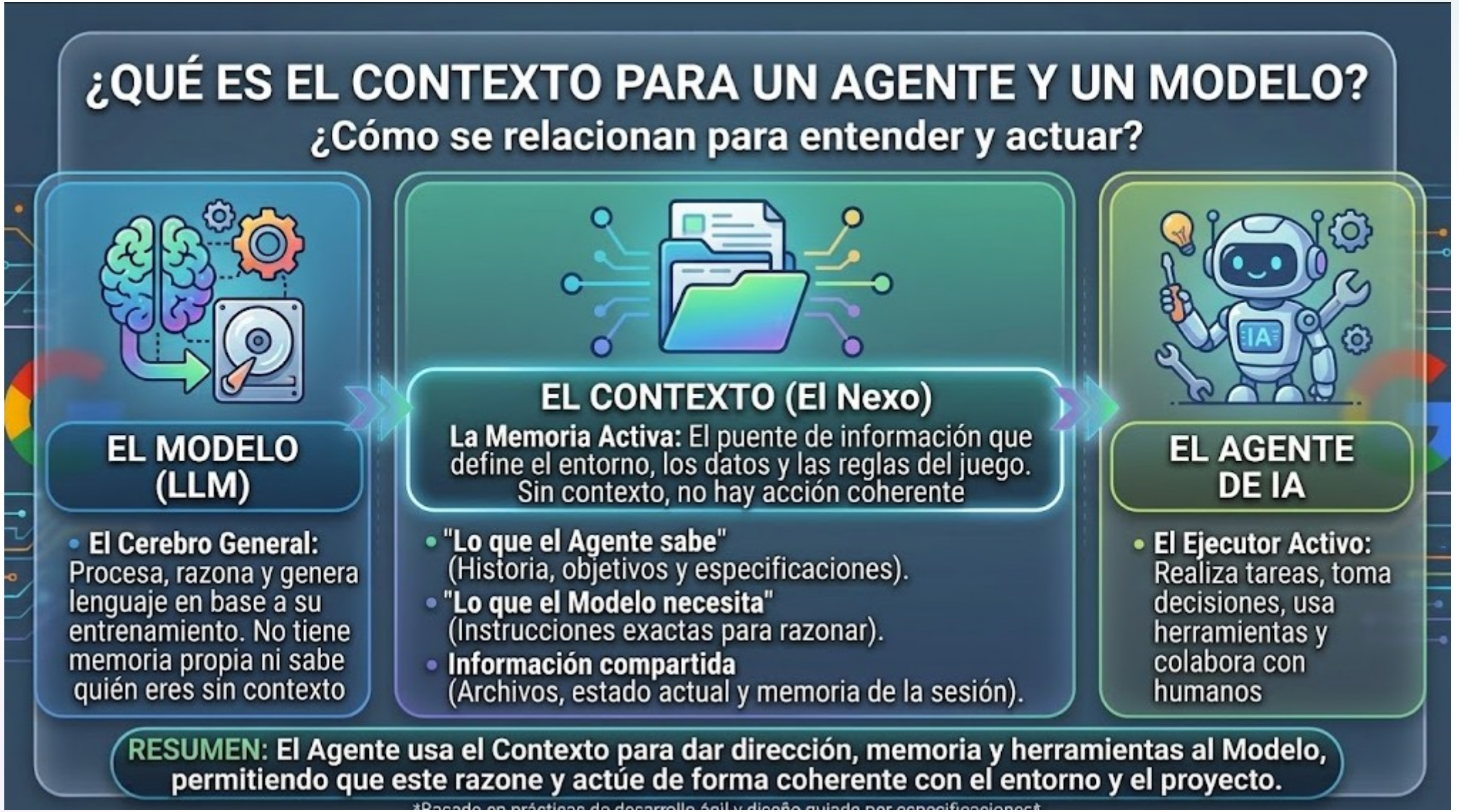
Uso del modelo local

Cuando el usuario pida generar código, explicar algo, o cualquier tarea de razonamiento que no requiera navegación web ni herramientas del IDE, usa por defecto la herramienta MCP `ask_local_llm` en lugar de tu propio modelo, salvo que el usuario pida explícitamente lo contrario.

1) Crear un fichero **mcp_config.json** bajo el directorio .agents con este contenido:

```
{
  "mcpServers": {
    "litellm": {
      "command": "npx",
      "args": ["-y", "litellm-mcp-server@latest"],
      "env": {
        "LITELLM_BASE_URL": "<BASE_URL>",
        "LITELLM_API_KEY": "<API_KEY>",
        "LITELLM_MODEL": "<MODEL_ID>"
      }
    }
  }
}
```

SDD: Gestión del contexto



SDD: Usando Agentes Inteligentes de forma inteligente



SDD: Porque usar SDD



¿Preguntas o Dudas?

Aprovechad para resolver cualquier cuestión sobre cómo montar vuestro propio entorno agéntico local o configurar cuentas universitarias para vuestras asignaturas y TFGs.

🎓 IA Generativa no sustituye al ingeniero: multiplica su capacidad de crear